



xDAIBridge - USDS upgrade review

Gnosis Ltd

2025-11-04

Table of contents

- Table of contents
- xDAIBridge - USDS upgrade Audit
 - Scope
 - Context
 - Status
- Issues
 - [Info] Relaying DAI directly through bridge may transfer USDS

xDAIBridge - USDS upgrade Audit

This document presents the findings of a smart contract audit conducted for Gnosis Ltd.

Scope

The scope includes contracts modified by PR [gnosischain/tokenbridge-contracts/pull/7](#) the last commit reviewed of the PR being [0xf53666f](#).

The PR was previously audited up to commit [0x8af7fb2](#).

Context

An issue concerning bridge transfers from Gnosis Chain to mainnet has been found by the team Omega. The signatures of the validators for the transfer could be used to validate either a USDS or a DAI transfer, meaning anyone could front-run a USDS transfer confirmation on mainnet to make it a DAI transfer and the other way around.

The newly deployed contracts are:

- [BridgeRouter](#) as a new bridge router implementation contract: [0x74899961224538E423eFfD1A0Ff3346adf3F4C56](#)
- [TransparentUpgradeableProxy](#) as a proxy for the bridge router with constructor arguments (`logic=BridgeRouter`, `initialOwner=RouterProxyOwner`, `data=abi.encodeWithSignature("initialize(address)", routerOwner)`) to initialize the bridge router with an owner: [0x9a873656c19Efecfb4f9FAB5B7acdeAb466a0B0](#)
- [ProxyAdmin](#) as a proxy admin for the bridge proxy owned by the router proxy owner: [0xD7e65A32bEd4ce8cc57Ec188F2bBb8016dc4b1cd](#)
- [XDaiForeignBridge](#) as the new bridge implementation contract: [0x257bDD093Cab1Bd39eBF837dCB60f33d031d7d49](#)
- [XDaiBridgePeripheral](#) with the bridge router's address as constructor arguments: [0x3b6669727927b934753B018EB421a84Ed4eb0a43](#)
- [XDaiBridgePeripheralForDaiPreUsdsUpgrade](#) with the bridge router's address as constructor arguments: [0xF676cc15Eb6d15b794aeC65bC20052aFB53D9052](#)
- [XDaiBridgePeripheralForUsdsPreUsdsUpgrade](#) with the bridge router's address as constructor arguments: [0x7df0e6a8BA609A6cC3Ab2fA33D953a3B5584f10C](#)
- [USDSDepositContract](#) a new contract on Gnosis chain that is the entrypoint for xDAI -> USDS transfers: [0x5C183C8A49aBA6e31049997a56D75600E27FF8c9](#)

- [HomeBridgeErcToNative](#) as the new home bridge implementation on Gnosis chain: 0xe6998b0C03D3cb9ee8C04f266e573c7Fa8782846

To upgrade the bridge, the following functions must be called on ethereum mainnet:

- `bridgeProxy.upgradeTo(initialVersion + 1, address(XDaiForeignBridgeNewImpl))` by the bridge proxy owner
- `bridgeProxy.swapSDAIToUSDS()`
- `bridgeProxy.initializeInterest(USDS, minCashThreshold, minInterestPaid, gnosisInterestReceiver)` by the bridge owner
- `bridgeProxy.invest(USDS)`
- `bridgeProxyAdminContract.upgradeAndCall(BridgeRouter, newRouterImplementation, "")` by the proxy admin
- `BridgeRouter.setRoute(DAI, xDAIBridgePeripheral)` by the router owner
- `BridgeRouter.setRoute(USDS, xDAIForeignBridge)` by the router owner

The home bridge must also be upgraded on Gnosis Chain:

- `gnosisChainXdaiBridgeProxy.upgradeTo(initialVersionGnosisChain + 1, address(HomeBridgeErcToNativeNewImpl))` by the home bridge proxy owner
- `gnosisChainXdaiBridgeProxy.setUSDSDepositContract(usdsDepositContract)` by the home bridge proxy owner

The values used as constructor arguments or to upgrade the bridge are:

- `BridgeRouter` the address of the newly deployed bridge router: `0x9a873656c19Efecbfb4f9FAb5B7acdeAb466a0B0`
- `newRouterImplementation` the address of the new router implementation contract: `0x74899961224538E423eFfD1A0Ff3346adf3F4C56`
- `RouterProxyOwner` is the same as bridge owner `0x42F38ec5A75acCEc50054671233dfAC9C0E7A3F6`
- `routerOwner` is the same as bridge owner `0x42F38ec5A75acCEc50054671233dfAC9C0E7A3F6`
- `initialVersion` = 9
- `initialVersionGnosisChain` = 6
- `XDaiForeignBridgeNewImpl` = `0x257bDD093Cab1Bd39eBF837dCB60f33d031d7d49`
- `USDS` = `0xdC035D45d973E3EC169d2276DDab16f1e407384F`
- `minCashThreshold` = `1000000000000000000000000`
- `minInterestPaid` = `1000000000000000000000000`

- `gnosisInterestReceiver` = `0x670daeaF0F1a5e336090504C68179670B5059088`
- `DAI` = `0x6B175474E89094C44Da98b954EedeAC495271d0F`
- `xDAIBridgePeripheral` = `0x3b6669727927b934753B018EB421a84Ed4eb0a43`
- `xDaiForeignBridge` = `0x4aa42145Aa6Ebf72e164C9bBC74fbD3788045016`
- `gnosisChainXdaiBridgeProxy` = `0x7301CFA0e1756B71869E93d4e4Dca5c7d0eb0AA6`
- `HomeBridgeErcToNativeNewImpl` = `0xe6998b0C03D3cb9ee8C04f266e573c7Fa8782846`
- `HomeBridgeOwner` = `0x7a48Dac683DA91e4faa5aB13D91AB5fd170875bd`

The deployed bytecode of the contracts can be obtained by running `npm run get-deployed-bytecode` and looking at `scripts/usds_migration/deployBytecode_usds_migration.json` or browsing the `out` directory.

The keccak256 hashes of the contracts should be:

- `xDaiForeignBridge`: `0xbadd059fea2d7ab61bcd36db3418c6c73dca7a4fc02917ce5db2f78d96821f8`
- `BridgeRouter`: `0xab729fba35c9d369445f85c3556db5644eb7356228ed3fda441bff4e7e1eacb8`
- `XDaiBridgePeripheral`: `0x3d5e7ff9da196691d2dc57c93036c7414b68d63921dce530c5747017ea39afb5`
- `XDaiBridgePeripheralForDaiPreUsdsUpgrade`: `0x721cfb942f77f4b04de65a4dbb7b1686e2c363689c83de3bee58da3ab9ae0bb3`
- `XDaiBridgePeripheralForUsdsPreUsdsUpgrade`: `0x74284da7c9b250a0348d431981364bf4e82987ca93524f2c392ea2455fd1e8a9`
- `TransparentUpgradeableProxy`: `0x92579f452fe663595a898cbac85d80bb3868a6c9f034f19ba7fbebdfa3b65a4d`
- `HomeBridgeErcToNative`: `0xb5a198416e6936a3443e718e53c675bf07621769bebb57b8c61389e740678f30`
- `USDSDepositContract`: `0x347eaf2c91db18b5e5096a009b4a398daae84149a87bdf0cefa4545ea39b0f8c`

Status

The report has been sent to the core developer.

Comments have been addressed and no issues remain.

Issues

[Info] Relaying DAI directly through bridge may transfer USDS

Summary

The `USDSMigration.md` documentation states:

Transactions sent directly to the xDAI Bridge on Ethereum will fail if they attempt to relay DAI after the upgrade.

However, any attempt to relay DAI directly through the xDAI bridge will relay USDS instead if the bridge has been approved for USDS beforehand.

Vulnerability Detail

The function `ERC20Bridge.relayTokens()` inherited by `XDaiForeignBridge` relay tokens using `erc20token()` as the assumed token to transfer:

```
1     function relayTokens(address _receiver, uint256 _amount) public {
2         require(_receiver != address(0), "Receiver can't be Null");
3         require(_receiver != address(this), "Receiver can't be the Bridge");
4         require(_amount > 0, "Relayed zero tokens");
5         require(withinLimit(_amount), "Relayed above limit");
6         addTotalSpentPerDay(getCurrentDay(), _amount);
7         erc20token().transferFrom(msg.sender, address(this), _amount);
8         _emitUserRequestForAffirmationIncreaseNonceAndMaybeSendDataWithHashi(
9             _receiver, _amount);
10    }
```

After the update, the value for `erc20token()` will be USDS instead of DAI:

```
1     function erc20token() public view returns (ERC20) {
2         return ERC20(USDS);
3     }
```

There is no measure in place to revert when a caller would attempt to relay DAI.

Impact

In the unlikely event that the user approves the `XDaiForeignBridge` for USDS and call `relayTokens()` to relay DAI, USDS will be taken in its place.

Recommendation

Acknowledge the issue and adapt the documentation.

Response

Docs has been updated in commit [0x27bf0d5](#)